Лабораторна робота №2

Студент: Бабенко Олександр, 402-тн

Проєктування архітектури програмного забезпечення та моделювання даних
Предмет: Технології розробки корпоративних Web-додатків

Обраний технологічний стек

Back-end: Node.js + NestJS
Front-end: React
База даних: PostgreSQL
ORM: Prisma
Аутентифікація: JWT
DevOps / Hosting: Docker, GitHub Actions
Документація API: Swagger

Обґрунтування вибору

Для реалізації інформаційної системи бібліотеки університету було обрано сучасний технологічний стек, який забезпечує масштабованість, надійність та зручність розробки.

Серверна частина реалізується на **Node.js з використанням NestJS**, оскільки система передбачає обробку значної кількості запитів (пошук книг, бронювання, робота з користувачами). NestJS дозволяє організувати код у вигляді модулів (auth, users, books, reservations), що спрощує підтримку та розширення системи. Використання TypeScript підвищує надійність за рахунок типізації.

Клієнтська частина реалізується за допомогою **React**, що дозволяє створити інтерактивний інтерфейс (пошук у реальному часі, особистий кабінет, адмін-панель) без перезавантаження сторінки. Компонентний підхід спрощує повторне використання елементів інтерфейсу.

Як СКБД обрано **PostgreSQL**, оскільки система має чітко структуровані дані та складні зв’язки (користувачі, книги, автори, видачі, бронювання). Реляційна модель дозволяє забезпечити цілісність даних і уникнути помилок.

Для взаємодії з БД використовується **Prisma ORM**, що забезпечує типобезпечні запити, зручну роботу з моделями та автоматичні міграції.

Для розгортання використовується **Docker**, що дозволяє стандартизувати середовище, а **GitHub Actions** — автоматизувати збірку та тестування.

Для контейнеризації та автоматизації обрано **Docker** і **GitHub Actions**. Docker дозволяє уніфікувати середовище запуску, а GitHub Actions — автоматизувати перевірку та збірку проєкту. Це зручно для командної розробки й подальшого розгортання.

2. Архітектурне моделювання за C4

2.1. Рівень 1 — System Context Diagram

Опис

На рівні контексту система розглядається як єдина чорна скринька. Показуються основні користувачі та зовнішні сервіси, які взаємодіють із системою.

Учасники системи

- Студент
- Викладач
- Бібліотекар
- Адміністратор
- Email-сервіс для надсилання сповіщень

Текстовий опис діаграми

- Студент використовує систему для пошуку, бронювання та перегляду історії позичань.
- Викладач використовує систему для пошуку та бронювання навчальної літератури.
- Бібліотекар використовує систему для оформлення видачі й повернення книг.
- Адміністратор керує користувачами, ролями та переглядає звіти.
- Система взаємодіє із зовнішнім Email-сервісом для надсилання нагадувань і повідомлень.

2.2. Рівень 2 — Container Diagram

Опис

На рівні контейнерів система деталізується на основні технічні блоки.

Контейнери системи

1. **Web Application (React SPA)**
   Вебінтерфейс для користувачів і персоналу.
2. **API Application (NestJS)**
   Серверна частина, яка містить бізнес-логіку, аутентифікацію, обробку запитів і доступ до даних.
3. **PostgreSQL Database**
   Реляційна база даних для зберігання інформації про користувачів, книги, бронювання та видачі.
4. **Email Notification Service**
   Зовнішній сервіс для відправлення листів.

Протоколи взаємодії

- React SPA ↔ API: HTTPS/REST
- API ↔ PostgreSQL: TCP/IP
- API ↔ Email Service: HTTPS/API

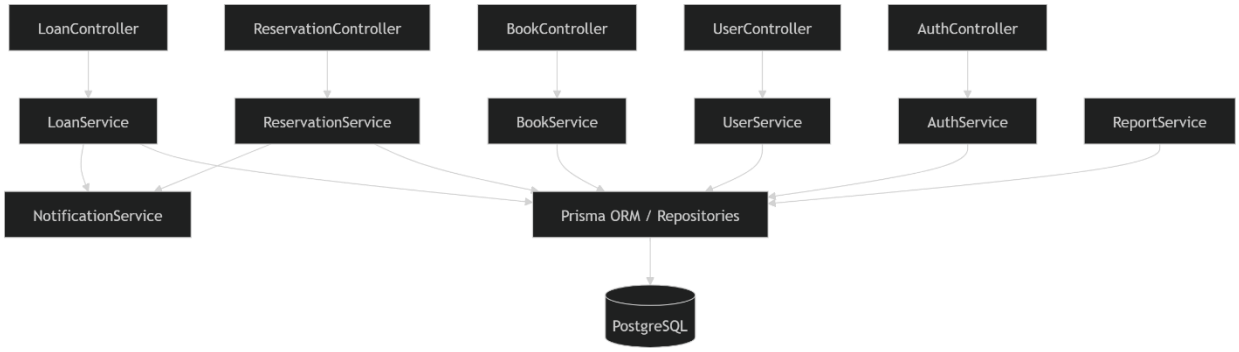
2.3. Рівень 3 — Component Diagram

Опис

На цьому рівні деталізується контейнер **API Application**.

Основні компоненти API

- **AuthController / AuthService** — реєстрація, вхід, JWT-аутентифікація
- **UserController / UserService** — керування користувачами
- **BookController / BookService** — пошук книг, перегляд каталогу
- **ReservationController / ReservationService** — бронювання книг
- **LoanController / LoanService** — видача та повернення книг
- **ReportService** — формування звітів
- **NotificationService** — надсилання email-сповіщень
- **Repositories / Prisma ORM** — доступ до бази даних



3. Проєктування бази даних (ER-модель)

Оскільки для системи обрано **PostgreSQL**, використовується **реляційна модель даних**.

Основні сутності

- User
- Role
- Book
- Author
- Category
- BookAuthor
- Reservation
- Loan

3.1. Сутності та атрибути

Role

- **role_id** (PK)
- name

User

- **user_id** (PK)
- full_name

- email
- password_hash
- role_id (FK → Role.role_id)
- created_at
- status

Category

- **category_id** (PK)
- name
- description

Author

- **author_id** (PK)
- full_name

Book

- **book_id** (PK)
- title
- isbn
- publication_year
- total_copies
- available_copies
- category_id (FK → Category.category_id)

BookAuthor

- **book_id** (PK, FK → Book.book_id)
- **author_id** (PK, FK → Author.author_id)

Reservation

- **reservation_id** (PK)
- user_id (FK → User.user_id)
- book_id (FK → Book.book_id)

- reservation_date
- status

Loan

- **loan_id** (PK)
- user_id (FK → User.user_id)
- book_id (FK → Book.book_id)
- issue_date
- due_date
- return_date
- status

3.2. Типи зв'язків

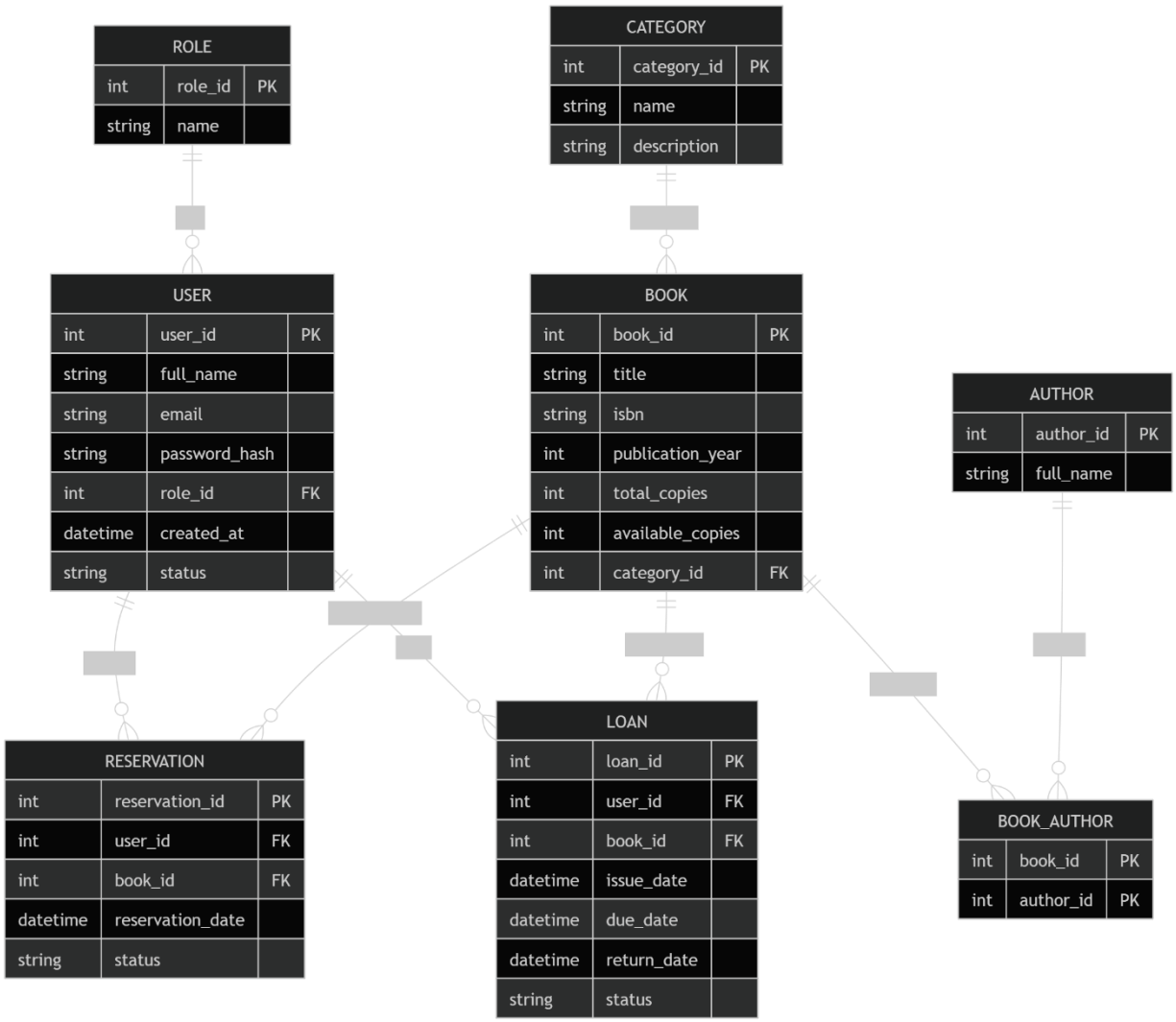
- Role → User = **1:N**
- Category → Book = **1:N**
- Book ↔ Author = **M:N** через BookAuthor
- User → Reservation = **1:N**
- User → Loan = **1:N**
- Book → Reservation = **1:N**
- Book → Loan = **1:N**

3.3. Обґрунтування нормалізації

Модель приведена щонайменше до **третьої нормальної форми (3НФ)**:

- дані про авторів винесені в окрему таблицю;
- багато-до-багатьох між книгами та авторами реалізовано через проміжну таблицю;
- ролі користувачів винесені в окрему сутність;
- дублювання інформації мінімізовано.

3.4. ER-діаграма



Висновок

У ході виконання лабораторної роботи було обрано технологічний стек для реалізації інформаційної системи бібліотеки університету та наведено його обґрунтування. Було спроєктовано архітектуру системи за методологією C4 на трьох рівнях: контекст, контейнери та компоненти, що дозволило послідовно описати систему від загального рівня до внутрішньої структури її основних модулів. Також було розроблено ER-модель бази даних, визначено основні сутності, атрибути, ключі та зв’язки між ними. Отримані результати

створюють основу для подальшої реалізації програмного продукту та забезпечують зрозуміле уявлення про його архітектуру і модель даних.

Відповіді на контрольні запитання

1. Що таке модель C4 і для чого потрібен кожен з її рівнів?

C4 Model — це підхід до опису архітектури програмного забезпечення за допомогою кількох рівнів деталізації.

- Context — показує систему в оточенні користувачів і зовнішніх сервісів.
- Container — показує основні технічні частини системи: клієнт, сервер, базу даних тощо.
- Component — деталізує внутрішню структуру одного контейнера.
- Іноді ще додають Code, але в лабораторній потрібні перші три рівні.

Ця модель потрібна для того, щоб архітектуру було зрозуміло як бізнесу, так і розробникам.

2. У чому різниця між SQL та NoSQL базами даних?

SQL-бази даних мають реляційну структуру, таблиці, зв’язки, зовнішні ключі та добре підходять для даних із чіткою схемою.

NoSQL-бази даних зазвичай мають гнучкішу структуру, наприклад документи JSON, і краще підходять для слабкоструктурованих або швидкозмінних даних.

SQL варто обирати, коли:

- важлива цілісність даних;
- є складні зв’язки;
- потрібні транзакції.

NoSQL варто обирати, коли:

- структура даних часто змінюється;
- потрібна висока горизонтальна масштабованість;
- дані зручніше зберігати документами.

3. Що таке теорема CAP?

CAP-теорема стверджує, що в розподіленій системі неможливо одночасно повністю забезпечити:

- Consistency — узгодженість даних;
- Availability — доступність;
- Partition Tolerance — стійкість до мережевих розривів.

На практиці при наявності мережевих проблем система зазвичай обирає компроміс між узгодженістю і доступністю. Це впливає на вибір бази даних: одні системи більше орієнтовані на строгість даних, інші — на постійну доступність.

4. Переваги і недоліки мікросервісної архітектури порівняно з монолітною

Переваги мікросервісів:

- окреме масштабування сервісів;
- незалежне розгортання;
- кращий поділ відповідальності;
- зручніше для великих команд.

Недоліки мікросервісів:

- складніше розгортання;
- складніша взаємодія між сервісами;
- важчий моніторинг і відлагодження;
- вищі вимоги до DevOps.

Монолітна архітектура простіша на старті, легше розробляється і підходить для невеликих або навчальних проєктів. Для цієї лабораторної доцільно обрати саме монолітну модульну архітектуру, оскільки вона простіша для реалізації, але при цьому дозволяє якісно спроєктувати систему.